
„float

HTWK

Hochschule für Technik,
Wirtschaft und Kultur Leipzig

FAKULTÄT INGENIEURWISSENSCHAFTEN

PROJEKT SEIKO DRUCKER

ELEKTRO UND INFORMATIONSTECHNIK (KOOPERATIV)

Belegarbeit

Autoren

Jonathan Blänsdorf, Paul Freiberger

Hochschul Betreuer

Henning Nagel, Ingo Reinhold

Leipzig, 9. September 2026

Inhaltsverzeichnis

1	Einführung	1
2	Ausgangssituation	2
2.1	Aktuellen Ist-Zustand beschreiben	2
2.2	Problem oder Bedarf klar darstellen	2
2.3	Rahmenbedingungen (technisch / organisatorisch) nennen	2
3	Aufgabenstellung	3
3.1	Konkrete Aufgaben und Ziele formulieren	3
3.2	Abgrenzung: Was ist Teil der Arbeit, was nicht	3
4	Struktur der Dokumentation	4
4.1	Gliederung der Arbeit erklären	4
4.2	Zusammenhang der Kapitel kurz begründen	4
5	Projektablauf	5
5.1	Vorgehensweise / Methodik beschreiben	5
5.2	Wichtige Arbeitsschritte chronologisch darstellen	7
5.3	Eingesetzte Werkzeuge oder Methoden erklären	8
6	Lösung	9
6.1	Erweiterungsmodule und Parameter	9
6.2	Hilfsfunktionen aufstellen	10
6.3	Schrittfunktion Definition	12
6.4	Startabfragen	17
7	Bedienung	18
8	Zusammenfassung und Ausblick	19
9	Anhang	20
	Quellenverzeichnis	27

Abbildungsverzeichnis	27
Tabellenverzeichnis	27

1 Einführung

Thema einordnen und Relevanz begründen

Ziel der Arbeit klar benennen

Aufbau der Arbeit kurz erläutern

Im Zuge der fortschreitenden Digitalisierung nahezu aller Lebensbereiche könnte der Eindruck entstehen, physische Drucktechnik habe ihren Zenit überschritten und sei auf dem Weg zur Bedeutungslosigkeit, doch dieser Eindruck täuscht. Gerade weil digitale Systeme immer leistungsfähiger werden, wächst gleichzeitig der Bedarf an präzisen, materialvielfältigen und funktionalen Druckverfahren. Sei es in der Elektronikfertigung, der Medizintechnik, der Verpackungsindustrie oder der Forschung, physische Drucktechnik findet heutzutage immer mehr Anwendungen.

Die vorliegende Belegarbeit entstand im Rahmen eines studentischen Projekts, dessen Ziel es ist, ein System zu automatisieren, mit dem verschiedene Drucktechniken praktisch erprobt und miteinander verglichen werden können. Mit Hilfe dieser Vorrichtung erhielt die Gruppe Studierender die Möglichkeit eigene Möglichkeiten zur Automatisierung einer industrienahen Maschine zu entwickeln und umzusetzen. Dieser handlungsorientierte Ansatz ermöglicht nicht nur den Erwerb technischen Fachwissens, sondern schärft zugleich das Verständnis für die Zusammenhänge zwischen Antriebstechnik, softwarebasierten Lösungsansätzen und industrieller Anwendung.

Die Relevanz dieses Projekts ergibt sich aus mehreren Gesichtspunkten: Zum einen wurde sich mit Antriebstechniken auseinandergesetzt, womit die im Studium erlernten Inhalte in der echten Welt angewendet werden konnten. Zum anderen mussten aufgrund der bereits vorhandenen Technik kreative, softwarebasierte Lösungsansätze ausgearbeitet werden, welche verschiedene Aspekte des Studiums vereinen. Zudem wurde durch die projektbezogene Zusammenarbeit eine klare und konstruktive Kommunikation gefördert, die wesentlich zum Projekterfolg beitrug.

2 Ausgangssituation

2.1 Aktuellen Ist-Zustand beschreiben

Am Anfang des Projekts wurde der Drucker bereits vollständig und betriebsbereit übergeben. Zudem wurde ein Laptop mitgeliefert, mit welchem die beiden Linearantriebe für x- und y-Achse angesteuert werden konnten. Diese Ansteuerung erfolgte allerdings nur händisch im Hand- bzw. Tippbetrieb. Somit befand der Drucker in einem nicht automatisierten Zustand.

2.2 Problem oder Bedarf klar darstellen

2.3 Rahmenbedingungen (technisch / organisatorisch) nennen

Dem Drucker beigelegt war eine Dokumentation und eine Betriebsanleitung. Die Dokumentation war allerdings in Japanisch geschrieben, was eine erste Herausforderung in der Inbetriebnahme des Druckers darstellte, weil Japanisch aufgrund der anderen Schriftzeichen und der vielfältigen Interpretationsmöglichkeiten dieser schwierig zu übersetzen ist.

Die Organisatorischen Rahmenbedingungen wie zeitliche Planung und Kostenmöglichkeiten wurden von den Hochschulbetreuern vorgegeben. Dabei wurde klargestellt, dass finanziell nur auf Restbestände von Laboren oder anderen Einrichtungen zurückgegriffen werden konnte. Außerdem wurde mitgeteilt, dass das Projekt keinen sonderlichen Zeitdruck hatte, allerdings mussten die Studierenden spätestens Ende des Sommersemesters 2026 dieses Projekt abschließen, weil in diesem Zeitraum das Studium der Studierenden zu Ende gehen würde. Laptop mit Windows 8 Die 2 Linearantriebe mit den Steuerantrieben/Encodern

3 Aufgabenstellung

3.1 Konkrete Aufgaben und Ziele formulieren

In einer E-Mail der Betreuenden wurde folgende Aufgabenstellung definiert:

Die Steuerung soll automatisiert ablaufen anstatt einen Bediener die Abläufe vollständig ausführen zu lassen. Die Nachrüstung soll möglichst kostengünstig mit dem bestehenden Drucker und dem zugehörigen Laptop ausgeführt werden. Die Abläufe sollen für verschiedene Drucktechniken individuell angepasst werden können. Eine Anleitung für die Arbeit mit der Automatisierungslösung soll mit einer übersichtlichen Projektdokumentation zusammen bereitgestellt werden. Das Projekt soll den Hochschulbetreuern präsentiert und samt der Doku vollständig übergeben werden.

3.2 Abgrenzung: Was ist Teil der Arbeit, was nicht

Erwartete Ergebnisse benennen

Das System soll den Druckkopf-Träger nach Vollendung steuern und über die Druckfläche führen. Die Bedienung soll sich grundlegend auf die Auswahl und ausführung von Programmen beziehen und nicht das Ausführen einzelner Steuerschritte. Die verfügbaren Ressourcen bei der Bearbeitung beschränken sich auf die verfügbaren Bauteile des Automatisierungslabors der HTWK Leipzig, Neubeschaffungen sind in dem Budget nicht vorgesehen.

4 Struktur der Dokumentation

4.1 Gliederung der Arbeit erklären

Vorgehen bei Projekt erklären um Nachvollziehbarkeit zu wahren Lösung vorstellen um späterer Bearbeitung zu ermöglichen

4.2 Zusammenhang der Kapitel kurz begründen

5 Projektablauf

5.1 Vorgehensweise / Methodik beschreiben

Der Ablauf des Projektes hat sich Thematisch in 7 verschiedene Etappen eingliedern lassen.

1. Projektentscheidung und erste Sichtung

Um tieferes Verständnis über die Funktion des bestehenden Druckers zu erlangen werden die mitgelieferte Dokumentationen gelesen. mit dabei liegt auch eine Anleitung zu Inbetriebnahme der grundlegenden Funktion und Steuerung.

klärung der Aufgabenstellung

inbetriebnahme des Druckers

ca. 5 Stunden

2. Automatisierungsmethode suchen

Ein schnell ersichtliches Problem war dass auf grund des Ursprungs der Dokumentation die gesamten Dokumentationen auf Japanisch geschrieben ist und nur vereinzelte Textpassagen in den manual auf Englisch übersetzt wurden.

Es wurden im Bezug auf die Rahmenbedingungen mögliche Lösungsansätze aufgestellt die Automatisierung des Systems auf Ebene der Steuerung über der vorhandenen Feldebene (Linearantrieben und Steuerantriebe) anzusetzen. Die erste aufgestellte Ansatz war die Umsetzung der Steuerung mit Hilfe einer Speicher Programmierbaren Steuerung. Die SPS hat sich seit Jahrzehnten als gängigste Steuerung im Industriellen Automatisierung Umfeld etabliert SPS Steuerung Kommunikationsprotokoll nachbilden GUI Steuerung

ca. 20 Stunden

3. Machbarkeit erproben

SPS Steuerung gescheitert an der Kommunikationsschnittstellen Für die Umsetzung der Steuerung mittels einer SPS war eine Siemens ET200-SP zur Verfügung gestellt wurde. Dieses Modell der SPS wird auch typischer Weise in der Industrie eingesetzt für die Ansteuerung von Antrieben. Die zwei vorhandenen Antriebe zusammen mit den vorhandenen Steuerungsantrieben/Encoder basieren jedoch nicht auf den gleichen Kommunikationsprotokollen wie die SPS es anbietet. Für die Umsetzung mit der vorhandenen SPS wäre eine weiteres Übersetzungsmodul notwendig um die Kommunikation zu errichten. Die Notwendigkeit von einer Neuanschaffung für eine erhebliche Summe Geld lässt diese Lösung ausscheiden, würde in der Umsetzung sehr Aufwendig sein aber auch am nächsten an den Industrie Standard kommen.

Um die bisherige Kommunikation der zwischen Laptop und der Linearantriebe zu replizieren schien es nahbar die gesendeten Protokolle welche Positionierungs Daten von Steuerung und Antrieb austauschen, einfach über die USB Schnittstelle abzufassen, analysieren und selbst später als Protokolle zu schicken um die Positionierung der Antriebe vorzugeben. Die Software auf dem Laptop die für beide Antriebe die Kommunikation übernimmt, sendet und empfängt verschlüsselte Pakete über die Position der Antriebe. Die Verschlüsselten Pakete lassen sich nicht ohne weiteres entschlüsseln und beinhalten einen Heartbeat welcher von dem Kommunikationspartner aufrechterhalten werden muss. Schlussendlich scheitert diese Methode daran dass sich in der Industrie die Informationssicherheit gezielt gegen Fremdeinwirkung und Sabotage richtet.

Da sich die bisherigen Ansätze zwischen der Feldebene und Steuerungsebene wenig Erfolg bereitet haben. Eine Ansatz Möglichkeit basiert auf der Annahme dass der Ausgangszustand in dem der Drucker schon alle relevanten Funktionen aufweist die Automatisierung nur bisher schon mit der richtigen Eingabe innerhalb der beiden Antriebssoftwares ausführbar war. Die Ebene zwischen Mensch und Maschine wird typischer Weise nicht als Möglichkeit weiter zu Automatisieren, bietet jedoch in unserem Fall mit den gegebenen Rahmenbedingungen die Möglichkeit Steuerung nach einem bestimmten Ablauf zu konzipieren. Um einen Menschlichen Bediener mittels Software nachzubilden lohnt sich der Einsatz von Methoden

die bereits auf genau solche Anforderungen konzipiert sind und im gegebenen Rahmen des Projektes Platz finden. Um die Funktionen die vorher der Mensch übernommen hat ist für die Automatisierung ein Teil Computer Vision nötig welcher in der Software die nötigen Kennzeichen erkennen kann. Für das Betätigen von Interaktiven Feldern innerhalb der Antriebssoftware können die Eingabe Signal von Peripherie Elementen simuliert werden. ca. 10 Stunden

4. **Entscheidung für Automatisierungslösung**

Die Lösung muss im Laufe der Bearbeitung auch mit den Hochschulichen Betreuern abgeklärt werden. Die Lösungsansätze der Implementierung einer SPS, sowie die Nachbildung des Kommunikationsprotokolls übersteigen die vorgegebene Rahmenbedingung im Sinne des Budgets/ Ressourcen da beide Möglichkeiten zwar möglich sein, jedoch die Neuanschaffung von Übersetzungsmodulen verlangt. Der Ansatz der Nachbildung von Eingangs und Ausgangssignalen mittels Computer Vision und Simulierten Menschlichen Eingaben bietet auf Grundlage des Ist-Zustands des Drucker Systems die bereits funktionstüchtige Basis bietet. Mit beiden Hochschul Betreuern wurde sich geeinigt dass die Einhaltung der Rahmenbedingungen über den Anforderungen von Industriellen Standards ansteht und eine experimentelle sowie auch individuelle Lösung den Vorstellungen an das Projekt näher erreicht. Für die Automatisierung wird eine Programmierung in Python vorgesehen sowie einige Erweiterungsmodule die in Python implementiert werden können um so weiter Möglichkeiten der Computer Vision, Ablaufprogrammierung oder Eingaben Simulation zu gewähren. Das Erweiterungspaket "pyautogui" bietet die Möglichkeit anhand einer Bibliothek von vorher aufgenommenen Bildern einem Programm Bedingungen zu stellen wann es den Ablauf weiter führen darf. Die Bedingung die bei der Durchführung beachtet werden muss ist dass das Windows 8 Betriebssystem nicht mit den neuesten Versionen von Python sowie dessen Erweiterungsmodulen arbeiten kann. GUI Steuerung über Python/pyautogui

ca. 2 Stunden

5. **Durchführung**

Erstellung von 2 Programmen mit denen über wenige eingaben der Ablauf gesteuert werden kann. Mittels der Dokumentation von Python sowie Pyautogui lassen sich die Programmierfortschritt schnell erreichen. Es wurden für den gesamten ablauf 2 verschiedene Programme mit python erstellt.

- (a) Programm 1 Pfadplanung Für das erste Programm war vorhergesehen eine Möglichkeit zu schaffen verschiedene Abläufe über den Druckbereich. Es wird interaktive gestaltet sodass keine Änderung von Parametern im Programmcode erforderlich ist sondern bei der Ausführung abgefragt wird welche Systemwerte vorhanden sind und daraufhin wird die Datei erstellt die in der Antriebssoftware den Verlauf ändert.
- (b) Programm 2 Ablaufsteuerung Dieses Programm startet die Bedienung der Antriebssteuerung und fragt zu Beginn noch einige Punkte ab um eine Fehlerfreien Grundzustand vorrauszusetzen. Die auszuführenden Schritte sollen jeweils durch eine Bedingung von einander getrennt sein. Das können Zeitintervalle sein aber auch das Abwarten von bestimmten visuellen Anzeichen über den Zustand des Druckers.

ca. 5 Stunden

6. **Testen und Fehlerbehebung**

Die entstandenen Programme zur Steuerung nach Ablauf der Antriebe soll auch wiederholbar ablaufen können und auch ablaufsicher gegen zufällige Fehler gestaltet sein. Um solche Fehler wurde das Programm nach den Anforderungen an Funktion erweitert an die Anforderungen an Aspekte wie Benutzerfreundlichkeit sowie Benutzersicherheit. ca. 4 Stunden

7. **Dokumentation und Vorstellung**

Das Festhalten des Projektablaufes bietet sich an um ca. 10 Stunden

5.2 Wichtige Arbeitsschritte chronologisch darstellen

Lösungsentscheidung fällen

Warum Entscheidung gefällt?

Kostengründe Zeitgründe Anpassbarkeit...

5.3 Eingesetzte Werkzeuge oder Methoden erklären

TIA Portal SPS Ansteuerung Advanced USB Port Monitor by AGG Software

1. Python: Die universelle Sprache der Automatisierung Python ist eine interpretierte Hochsprache, die sich durch eine klare Syntax und eine enorme Vielseitigkeit auszeichnet. In der modernen Automatisierungstechnik (Industrie 4.0) hat Python ein herausragendes Standing als „Glue Language“ (Klebesprache) eingenommen. Sie dient als Bindeglied zwischen der physikalischen Feldebene und der Informationstechnik. Laut dem TIOBE-Index und dem IEEE Spectrum Ranking belegt Python regelmäßig Spitzenplätze als eine der weltweit am häufigsten genutzten Programmiersprachen. In diesem Projekt ermöglicht Python die Ablaufsteuerung, welche die Position der Linearantriebe koordiniert. py-admin 2026

2. pytime pysys

2. PyAutoGUI: Robotic Process Automation (RPA) auf GUI-Ebene PyAutoGUI ist eine spezialisierte Programmbibliothek zur Steuerung von Maus und Tastatur über Python-Skripte. Im Kontext Ihres Druckers fungiert PyAutoGUI als Werkzeug für die Robotic Process Automation (RPA). Da industrielle Druckersoftware oft proprietär ist und keine direkte Programmierschnittstelle (API) bietet, ermöglicht PyAutoGUI die Automatisierung des Ablaufs durch die Emulation menschlicher Interaktionen mit der grafischen Benutzeroberfläche (GUI) 3. Dies ist besonders wertvoll, um die X- und Y-Koordinatenbefehle in die Steuerungssoftware einzugeben, ohne den Quellcode der Software modifizieren zu müssen.

Quelle: PyAutoGUI Dokumentation (ReadTheDocs) und AI Sweigart: Automate the Boring Stuff with Python.

3. NumPy: Numerische Präzision für die Achsensteuerung NumPy (Numerical Python) ist das fundamentale Paket für wissenschaftliches Rechnen in Python. In Ihrem Projekt dient es als Erweiterung, um die mathematischen Berechnungen für die Linearantriebe effizient durchzuführen. Während Standard-Python-Listen für komplexe mathematische Operationen langsamer sind, bietet NumPy leistungsstarke N-dimensionale Arrays. Dies ist entscheidend für die Berechnung von Bewegungsbahnen, Geschwindigkeitsrampen oder Koordinatentransformationen der X- und Y-Achsen in Echtzeit-naher Geschwindigkeit.

Quelle: NumPy.org (official) sowie Harris, C. R., et al. (2020): Array programming with NumPy. In: Nature 5.

4. Thonny: Ein leichtgewichtiger Editor für Windows 8 Thonny ist eine integrierte Entwicklungsumgebung (IDE), die speziell für Einsteiger und ressourcenschonende Umgebungen entwickelt wurde. Die Wahl von Thonny ist für Ihr Projekt besonders sinnvoll, da das Betriebssystem Windows 8 geringere Systemressourcen beansprucht als modernere Versionen. Thonny zeichnet sich durch eine einfache Installation (inklusive integriertem Python-Interpreter) und einen hervorragenden Debugger aus, der die schrittweise Analyse der Ablaufsteuerung für die Druckerachsen erleichtert.

Quelle: Thonny.org (official) sowie das Repository von Aivar Annamaa (Universität Tartu).

6 Lösung

Die nach dem Testen und den Fehlerbehebungen entstandene Programme sind hier in Funktion erklärt.

6.1 Erweiterungsmodule und Parameter

```
1  import pyautogui
2  import time
3  import sys
4
5  # =====
6  # KONFIGURATION - Zeitlimits und Wartezeiten (in Sekunden)
7  # =====
8  CONFIG = {
9      "min_wait_pcon_step":      3.0, # Mindestlaufzeit nach pcon-Schritt
10     "max_wait_pcon_step":      10.0, # Maximallaufzeit für pcon-Schritt
11     "min_wait_mrconfig_step":  3.0, # Mindestlaufzeit nach MR Config-Schritt
12     "max_wait_switch_window":  10.0, # Maximallaufzeit für Fensterwechsel
13     "wait_after_opstart":      12.0, # Wartezeit nach opstartmrconfig
14     "wait_after_arrow":        3.0, # Wartezeit nach Pfeil-Klick
15     "search_confidence":       0.8, # Erkennungssicherheit für pyautogui
16 }
17
```

Abbildung 1: pyautogui Erweiterung und Programm Parameter

Die zu importierenden Pakete werden aufgerufen und die Parameter die innerhalb des Programmes benutzt werden, werden initialisiert. Diese Werte können hier angepasst werden, wenn die Abfolge der Schritte mit anderen Zeitparametern ablaufen sollen.

```
18 # =====
19 # CHECKLISTE
20 # =====
21 CHECKLIST = [
22     "Stecker eingesteckt",
23     "Handbetrieb eingestellt",
24     "MR Config gestartet",
25     "Positionsdatei in MR Config geladen",
26     "pCon Software gestartet",
27     "Positionsdatei in pCon geladen",
28 ]
```

Abbildung 2: Liste der Startbedingungen

Hier wird die Liste der Bedingungen festgehalten, die vor dem Start abgehakt werden müssen um das Programm aus einem gesicherten Zustand anfangen zu können.

6.2 Hilfsfunktionen aufstellen

```
30 # =====
31 # HILFSFUNKTIONEN
32 # =====
33
34 def find_and_click(image_path, timeout=10.0, min_wait=0.0, description=""):
35     """
36     Sucht ein Bild auf dem Bildschirm und klickt darauf.
37     Wartet mindestens min_wait Sekunden und maximal timeout Sekunden.
38     Gibt True zurück wenn erfolgreich, False wenn nicht gefunden.
39     """
40     start_time = time.time()
41     print(f" 🔍 Suche nach: {description or image_path}")
42
43     while time.time() - start_time < timeout:
44         try:
45             location = pyautogui.locateCenterOnScreen(
46                 image_path,
47                 confidence=CONFIG["search_confidence"]
48             )
49             if location:
50                 elapsed = time.time() - start_time
51                 # Mindestlaufzeit einhalten
52                 if elapsed < min_wait:
53                     time.sleep(min_wait - elapsed)
54                 pyautogui.click(location)
55                 print(f" ✅ Gefunden und geklickt: {description or image_path}")
56                 return True
57             except pyautogui.ImageNotFoundException:
58                 pass
59             time.sleep(0.5)
60
61     print(f" ❌ FEHLER: '{description or image_path}' nicht gefunden innerhalb von {timeout}s!")
62     return False
63
```

Abbildung 3: Funktion Screenshots finden

Die Funktion innerhalb von autogui die innerhalb des Programmes auf die im Vorhinein gewählten Bildschirmaufnahmen zugreift, diese innerhalb des derzeitigen Desktops sucht und mit dem Mauszeiger anklickt.

```
64 def is_on_screen(image_path):
65     """
66     Prüft ob ein Bild auf dem Bildschirm sichtbar ist (ohne zu klicken).
67     """
68     try:
69         location = pyautogui.locateOnScreen(
70             image_path,
71             confidence=CONFIG["search_confidence"]
72         )
73         return location is not None
74     except pyautogui.ImageNotFoundException:
75         return False
```

Abbildung 4: Funktion Anzeige prüfen

Hierbei handelt es sich um den Programmausschnitt in dem die Funktion von autogui beschrieben wird die auf dem Bildschirm die passende .png Datei sucht um zu kontrollieren ob sie derzeit auf dem Bildschirm abgebildet ist.

```

77 def ensure_pcon_active():
78     """
79     Stellt sicher, dass das pCon-Fenster aktiv ist.
80     """
81     print("\n 🖱️ Prüfe ob pCon-Fenster aktiv ist...")
82     if is_on_screen("currentwindowpcon.png"):
83         print(" ✅ pCon-Fenster ist bereits aktiv.")
84         return True
85     else:
86         print(" ⚠️ pCon nicht aktiv 🖱️ suche Icon in Taskleiste...")
87         success = find_and_click(
88             "pconicon.png",
89             timeout=CONFIG["max_wait_switch_window"],
90             description="pCon Taskleisten-Icon"
91         )
92         if not success:
93             print(" ❌ KRITISCH: pCon konnte nicht aktiviert werden!")
94             sys.exit(1)
95             time.sleep(1.0)
96             return True

```

Abbildung 5: Aktives PCon Fenster prüfen

In dieser Funktion wird geprüft ob das benötigte Fenster der PCon Applikation geöffnet ist.

```

98 def ensure_mrconfig_active():
99     """
100     Stellt sicher, dass das MR Config-Fenster aktiv ist.
101     """
102     print("\n 🖱️ Wechsle zu MR Config...")
103     if is_on_screen("currentwindowmrconfig.png"):
104         print(" ✅ MR Config-Fenster ist bereits aktiv.")
105         return True
106     else:
107         print(" ⚠️ MR Config nicht aktiv 🖱️ suche Icon in Taskleiste...")
108         success = find_and_click(
109             "mrconfigicon.png",
110             timeout=CONFIG["max_wait_switch_window"],
111             description="MR Config Taskleisten-Icon"
112         )
113         if not success:
114             print(" ❌ KRITISCH: MR Config konnte nicht aktiviert werden!")
115             sys.exit(1)
116             time.sleep(1.0)
117             return True

```

Abbildung 6: Aktives MR Configurator Fenster prüfen

In dieser Funktion wird geprüft ob das benötigte Fenster des MR-Configurator Applikation geöffnet ist.

6.3 Schrittfunktion Definition

```
119 # =====
120 # SCHRITT-FUNKTIONEN
121 # =====
122
123 def mrconfig_sequence():
124     """
125     SchrittMRConfig □ Die immer gleiche MR Config-Sequenz.
126     Wiederverwendbar in SchrittPcon0 und SchrittPcon.
127     Alle Pfeil- und OP-Start-Schritte haben Mindestlaufzeiten.
128     """
129     print("\n ⚙  Starte MR Config Sequenz...")
130
131     if not is_on_screen("currentwindowmrconfig.png"):
132         print(" ❌ MR Config Fenster nicht erkannt!")
133         sys.exit(1)
134
135     if not is_on_screen("pointtabelle01.png"):
136         print(" ❌ Punkttabelle Nr. 1 nicht gefunden!")
137         sys.exit(1)
138
139     # — Schritt 1: OP Start —————
140     if not find_and_click(
141         "opstartmrconfig.png",
142         timeout=CONFIG["max_wait_pcon_step"],
143         min_wait=CONFIG["min_wait_mrconfig_step"],
144         description="OP Start MR Config (1)"
145     ):
146         sys.exit(1)
147     time.sleep(CONFIG["wait_after_opstart"])
148
149     # — Schritt 2: Pfeil HOCH —————
150     if not find_and_click(
151         "pfeilhoch.png",
152         timeout=CONFIG["max_wait_pcon_step"],
153         min_wait=CONFIG["min_wait_mrconfig_step"],
154         description="Pfeil Hoch"
155     ):
156         sys.exit(1)
157     time.sleep(CONFIG["wait_after_arrow"])
158
```

Abbildung 7: Anfang der MR Configurator Schrittkette, MR Schritt 1 und 2

Die Schritte die der, durch den MR Configurator gesteuerte Linearantrieb, ausführen muss, sind identisch. Es müssen bei diesem Schritt nur 2 verschiedene Positionen angefahren werden, jedoch abhängig von der Spaltenanzahl mehrfach wiederholt werden.

```

159 # — Schritt 3: OP Start —————
160 if not find_and_click(
161     "opstartmrconfig.png",
162     timeout=CONFIG["max_wait_pcon_step"],
163     min_wait=CONFIG["min_wait_mrconfig_step"],
164     description="OP Start MR Config (2)"
165 ):
166     sys.exit(1)
167 time.sleep(CONFIG["wait_after_opstart"])
168
169 # — Schritt 4: Pfeil RUNTER —————
170 if not find_and_click(
171     "pfeilrunter.png",
172     timeout=CONFIG["max_wait_pcon_step"],
173     min_wait=CONFIG["min_wait_mrconfig_step"],
174     description="Pfeil Runter"
175 ):
176     sys.exit(1)
177 time.sleep(CONFIG["wait_after_arrow"])
178
179 # — Schritt 5: OP Start (letzter) —————
180 if not find_and_click(
181     "opstartmrconfig.png",
182     timeout=CONFIG["max_wait_pcon_step"],
183     min_wait=CONFIG["min_wait_mrconfig_step"],
184     description="OP Start MR Config (3)"
185 ):
186     sys.exit(1)
187 time.sleep(CONFIG["wait_after_opstart"])
188
189 print("  ✓ MR Config Sequenz abgeschlossen.")
190

```

Abbildung 8: MR Schritt 3,4 und 5

```

191 def schritt_pcon():
192     """
193     SchrittPcon0  Wird NUR EINMAL am Anfang ausgeführt.
194     Initialisierungsschritt: Erster Schritt + Erster Spaltenpunkt + MR Config.
195
196     ⚠ Schritt 1 (Schritt in pCon auswählen) wird NUR ausgeführt,
197     wenn pos0.png NICHT auf dem Bildschirm gefunden wird.
198     """
199     print(f"\n{' '*55}")
200     print(f" 🚀 SchrittPcon0  Initialisierung (einmalig)")
201     print(f"{' '*55}")
202
203     # 1. pCon aktivieren
204     ensure_pcon_active()
205
206     # — Schritt 1: NUR ausführen wenn pos0.png NICHT sichtbar —
207     print("\n 🔍 Prüfe ob Position 0 bereits aktiv (pos0.png)...")
208     if not find_and_click(
209         "schritt_pcon.png",
210         timeout=CONFIG["max_wait_pcon_step"],
211         min_wait=CONFIG["min_wait_pcon_step"],
212         description="Schritt in pCon auswählen (Schritt 1)"
213     ):
214         sys.exit(1)
215     else:
216         print(" ✅ pos0.png gefunden → Position bereits aktiv, Schritt 1 wird übersprungen.")
217
218     # — Schritt 2: Ersten Spaltenpunkt anfahren (IMMER) —
219     print("\n 🚀 Schritt 2: Ersten Spaltenpunkt anfahren...")
220     if not find_and_click(
221         "schritt_pcon.png",
222         timeout=CONFIG["max_wait_pcon_step"],
223         min_wait=CONFIG["min_wait_pcon_step"],
224         description="Erster Spaltenpunkt"
225     ):
226         sys.exit(1)
227
228     # Mindestlaufzeit vor Fensterwechsel
229     time.sleep(CONFIG["min_wait_pcon_step"])
230
231     # — Schritt 3: Zu MR Config wechseln —
232     ensure_mrconfig_active()
233
234     # — Schritt 4: MR Config Sequenz ausführen —
235     mrconfig_sequence()
236
237     # — Schritt 5: Zurück zu pCon —
238     ensure_pcon_active()
239
240     print(f"\n ✅ SchrittPcon0 abgeschlossen.")

```

Abbildung 9: Initialschritt unabhängig von der Spaltenanzahl

Der Initialschritt muss von der PCon Software ausgeführt werden um sicher zu stellen dass sich der Druckkopfträger auf der vorgesehenen Startposition bereitsteht. Danach kann die Abfolge der Schritte beginnen.

```

242 def schritt_pcon(spalten_nummer):
243     """
244     SchrittPcon  Wird für JEDE weitere Spalte wiederholt.
245     Führt den nächsten Spaltenpunkt an und führt MR Config Sequenz aus.
246     """
247     print(f"\n{' '*55}")
248     print(f"  SchrittPcon  Spalte {spalten_nummer}")
249     print(f"{' '*55}")
250
251     # 1. pCon aktivieren
252     ensure_pcon_active()
253
254     # 2. Nächsten Spaltenpunkt anfahren
255     print(f"\n  Spaltenpunkt {spalten_nummer} anfahren...")
256     if not find_and_click(
257         "schritt_pcon.png",
258         timeout=CONFIG["max_wait_pcon_step"],
259         min_wait=CONFIG["min_wait_pcon_step"],
260         description=f"Spaltenpunkt {spalten_nummer}"
261     ):
262         sys.exit(1)
263
264     # Mindestlaufzeit vor Fensterwechsel
265     time.sleep(CONFIG["min_wait_pcon_step"])
266
267     # 3. Zu MR Config wechseln
268     ensure_mrconfig_active()
269
270     # 4. MR Config Sequenz ausführen
271     mrconfig_sequence()
272
273     # 5. Zurück zu pCon
274     ensure_pcon_active()
275
276     print(f"\n  SchrittPcon für Spalte {spalten_nummer} abgeschlossen.")
277

```

Abbildung 10: PCon Schrittkette abhängig von der Spaltenanzahl des Testversuches

Die für den Testversuch gewählte Aufteilung der Druckoberfläche in Spalten, bestimmt innerhalb des Programmes wie häufig die Schrittfolge wiederholt werden muss.

```

278 def schritt_pcon_abschluss():
279     """
280     SchrittPconAbschluss ☐ Wird NUR EINMAL ganz am Ende ausgeführt.
281     Wiederholt einen letzten pCon-Schritt nach der finalen MR Config Sequenz.
282     """
283     print(f"\n{' '*55}")
284     print(f" ☒ SchrittPconAbschluss ☐ Letzter pCon-Schritt")
285     print(f"{' '*55}")
286
287     # 1. pCon aktivieren
288     ensure_pcon_active()
289
290     # 2. Letzten pCon-Schritt ausführen
291     print("\n 🚀 Letzten pCon-Schritt ausführen...")
292     if not find_and_click(
293         "schritt_pcon.png",
294         timeout=CONFIG["max_wait_pcon_step"],
295         min_wait=CONFIG["min_wait_pcon_step"],
296         description="Letzter pCon-Schritt (Abschluss)"
297     ):
298         sys.exit(1)
299
300     # Mindestlaufzeit nach letztem Schritt
301     time.sleep(CONFIG["min_wait_pcon_step"])
302
303     print(f"\n ✅ Abschlussschritt in pCon erfolgreich ausgeführt.")
304

```

Abbildung 11: PCon Schlussschritt unabhängig von Spaltenanzahl

Damit bei Wiederholungen der gesamten Schrittfolge die nötige Ausgangsstellung schneller erreicht werden kann, endet jede Schrittkette damit den Druckkopfträger wieder an die Ausgangsposition zurückzuführen.

6.4 Startabfragen

```
305 # =====
306 # CHECKLISTE ABFRAGEN
307 # =====
308
309 def run_checklist():
310     """
311     Fragt die Ausgangsbedingungen ab.
312     Gibt erst frei wenn alle Punkte bestätigt wurden.
313     Gibt die Anzahl der Spalten zurück.
314     """
315     print("\n" + "="*55)
316     print("  SEIKO DRUCKER AUTOMATISIERUNG  STARTCHECKLISTE")
317     print("="*55)
318     print("Bitte alle Punkte prüfen und mit 'j' bestätigen.\n")
319
320     for i, item in enumerate(CHECKLIST, start=1):
321         while True:
322             answer = input(f" [{i}/{len(CHECKLIST)}] {item}  Bereit? (j/n): ").strip().lower()
323             if answer == 'j':
324                 print(f"  ✅ Bestätigt: {item}\n")
325                 break
326             elif answer == 'n':
327                 print(f"  ⚠️ Bitte zuerst '{item}' erledigen und dann erneut bestätigen.\n")
328             else:
329                 print("  Bitte 'j' für Ja oder 'n' für Nein eingeben.")
330
```

Abbildung 12: Checkliste zur Bestätigung aufrufen

Die Checkliste wird ausgegeben und muss händisch für jede Bedingung bestätigt werden bevor das Programm anlaufen kann.

```
331 # Anzahl Spalten abfragen
332 while True:
333     try:
334         anzahl = int(input("\n Wie viele Spalten sollen verarbeitet werden? "))
335         if anzahl > 0:
336             print(f"  ✅ Anzahl Spalten: {anzahl}\n")
337             break
338         else:
339             print("  Bitte eine positive Zahl eingeben.")
340     except ValueError:
341         print("  Bitte eine gültige Zahl eingeben.")
342
343     print("\n✅ Checkliste vollständig  Automatisierung startet!\n")
344     return anzahl
345
```

Abbildung 13: Anzahl der Spalten abfragen

Die für den Testvorgang bestimmte Spaltenanzahl wird hier in einer Eingabefenster abgefragt.

7 Bedienung

Wie wird das Programm vorbereitet

Hardware:

Sichtprüfung des Innenraums (Fremdkörper entfernen)

Druckraumtüren schließen (Quetschgefahr während Ablauf verhindern)

CEE Stecker einstecken (Spannung anlegen)

Rep Schalter auf Maintenance (Handbetrieb) schalten

Starttaster (grün) betätigen

die beiden Linearantriebe sollten nun unter Spannung stehen und nehmen ihre Position auf

aus vorherigen Abläufen können die Linearantriebe auch die Druckplatte blockieren

um die Platte mit den Druckmedien zu bestücken können die Linearantriebe per Hand aus dem Weg geschoben werden

das Druckmedium kann mit betätigung des Tasters -P1 innen im Druckraum an die Druckplatte gesaugt werden aus dem Schaltschrank führen die Datenkabel der zwei Steuermodule

USB Kabel verbunden mit dem USB Hub in freie USB Schnittstelle stecken

den Laptop starten und sich mit dem Benutzer student anmelden

Software:

thonny den python Editor starten

Pythonprogramm 1 zur Erstellung des Spalten ablaufes starten

auf z.B. desktop abspeichern

MR Configurator starten

Positionierungsprogramm laden

PCon PC Interface

selbst erstelltes Positionierungsprogramm aus ablageort importieren

Pythonprogramm 2 zur Ablaufsteuerung ausführen Checkliste ab arbeiten Spaltenanzahl angeben

Programm startet automatisch Ausgangszustand herstellen (beide Antriebe auf Position 0 stellen)

8 Zusammenfassung und Ausblick

Das Ziel eines Automatisierten Systems konnte während des Projektverlaufes nach Zieldefinition und Festlegung auf Rahmenbedingungen erreicht werden. Durch die Absprache mit den Hochschulbetreuern waren die Erwartungen an das Projekt auch transparent gestellt sodass keine Missverständnisse durch das Projekt gezogen haben. Die Funktion die der Drucker durch das Implementieren der Python Programme gewonnen hat ermöglichen nun die Nutzung des Druckers für verschiedene Testzwecke im Feld der Lern- und Testprozesse.

Die Beleuchtung verschiedener Ansätze und die Analyse der Machbarkeit zeigt die entstandenen Herausforderungen und die Abwägung der Lösungen.

Mögliche Weiterentwicklungen oder offene Fragen aufzeigen

Basierend auf dieser Dokumentation ist es möglich auch weitere Funktionen dem Ablauf hinzu zu fügen. Die Erläuterung der Python Programme sollte dabei von Nutzen sein um Weiterentwicklungen wie das Repetitive ausführen des ganzen ablaufes anhand einer Eingabe oder das Abfahren einer Teilmenge der Druckoberfläche. Nötige Veränderungen könnten schnell Implementiert werden durch die Dokumentation des Programmes.

9 Anhang

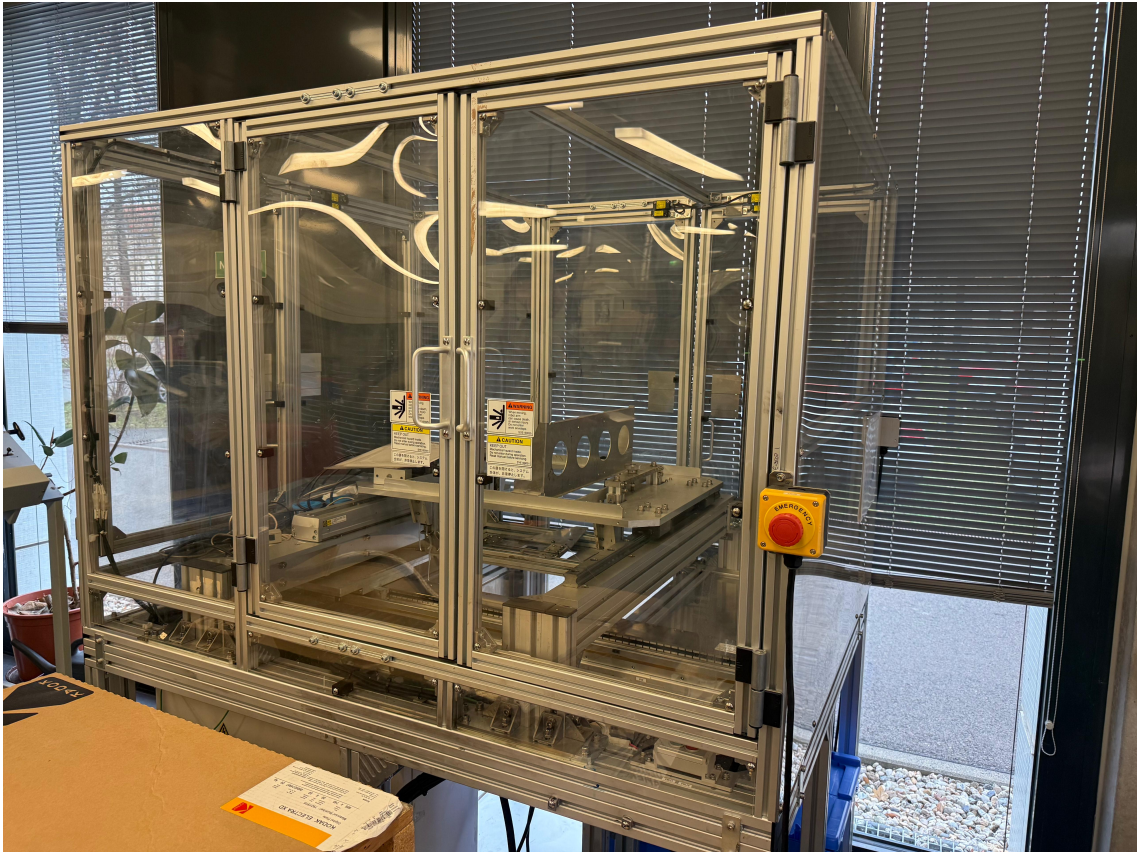


Abbildung 14: Gesamtansicht des Drucker Systems

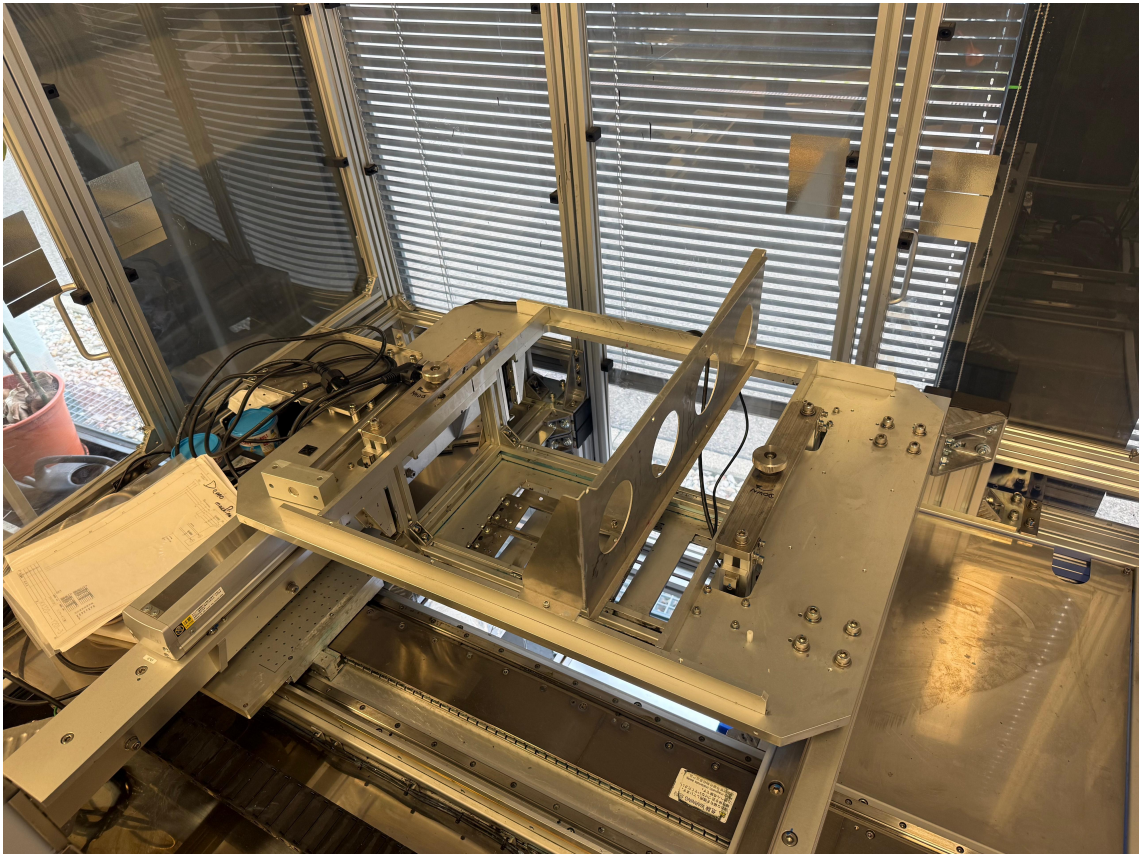


Abbildung 15: Bauraum des Drucker Systems



Abbildung 16: Rückseite, Linearantrieb IAI (von PCon gesteuert)



Abbildung 17: Vorderseite, Linearantrieb IAI (von PCon gesteuert)



Abbildung 18: Rückseite, Linearantrieb von Mitsubishi (von MR Configurator gesteuert)

3 Spalten

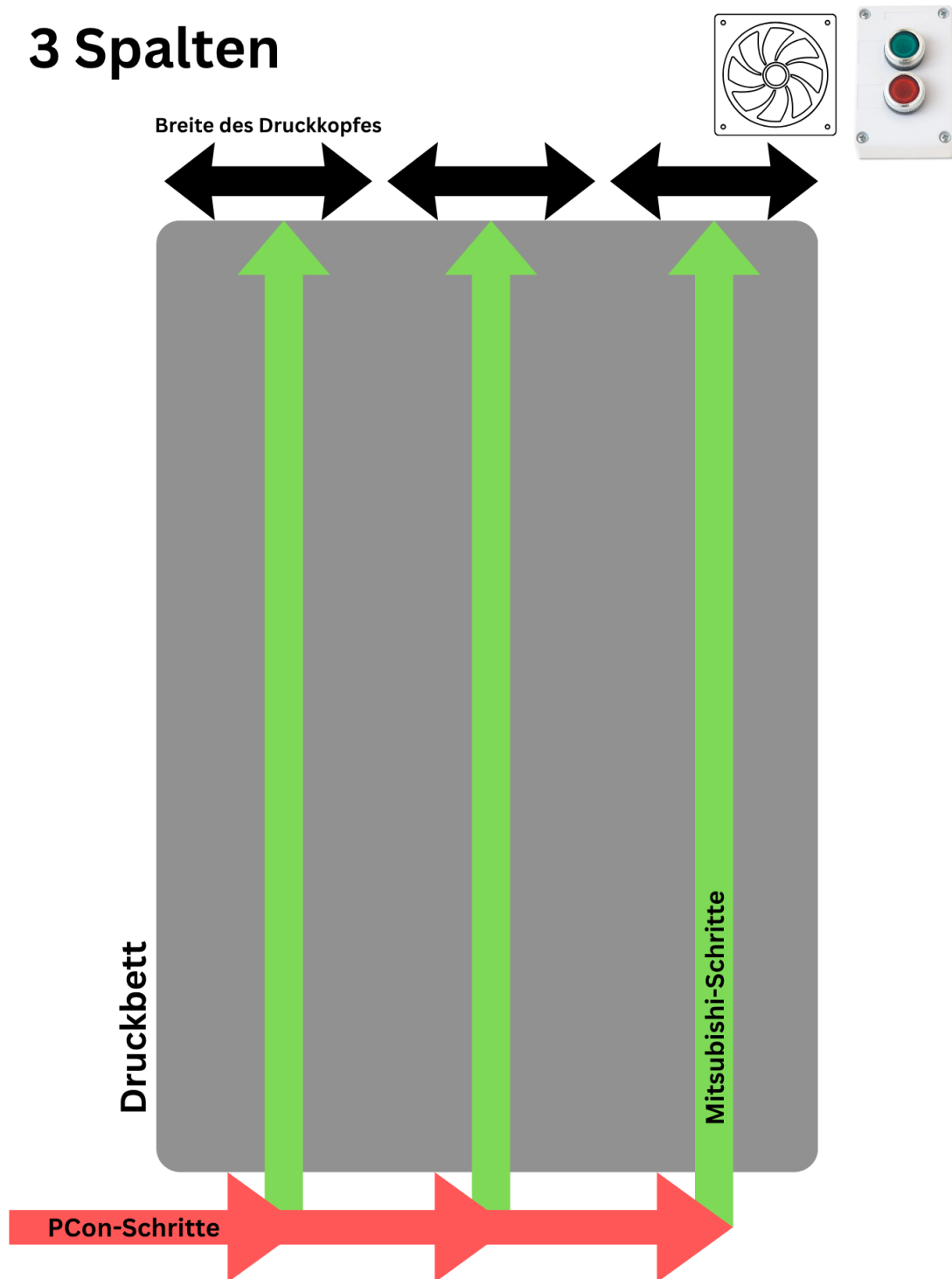


Abbildung 19: Ablauf bei 3 Spalten

5 Spalten

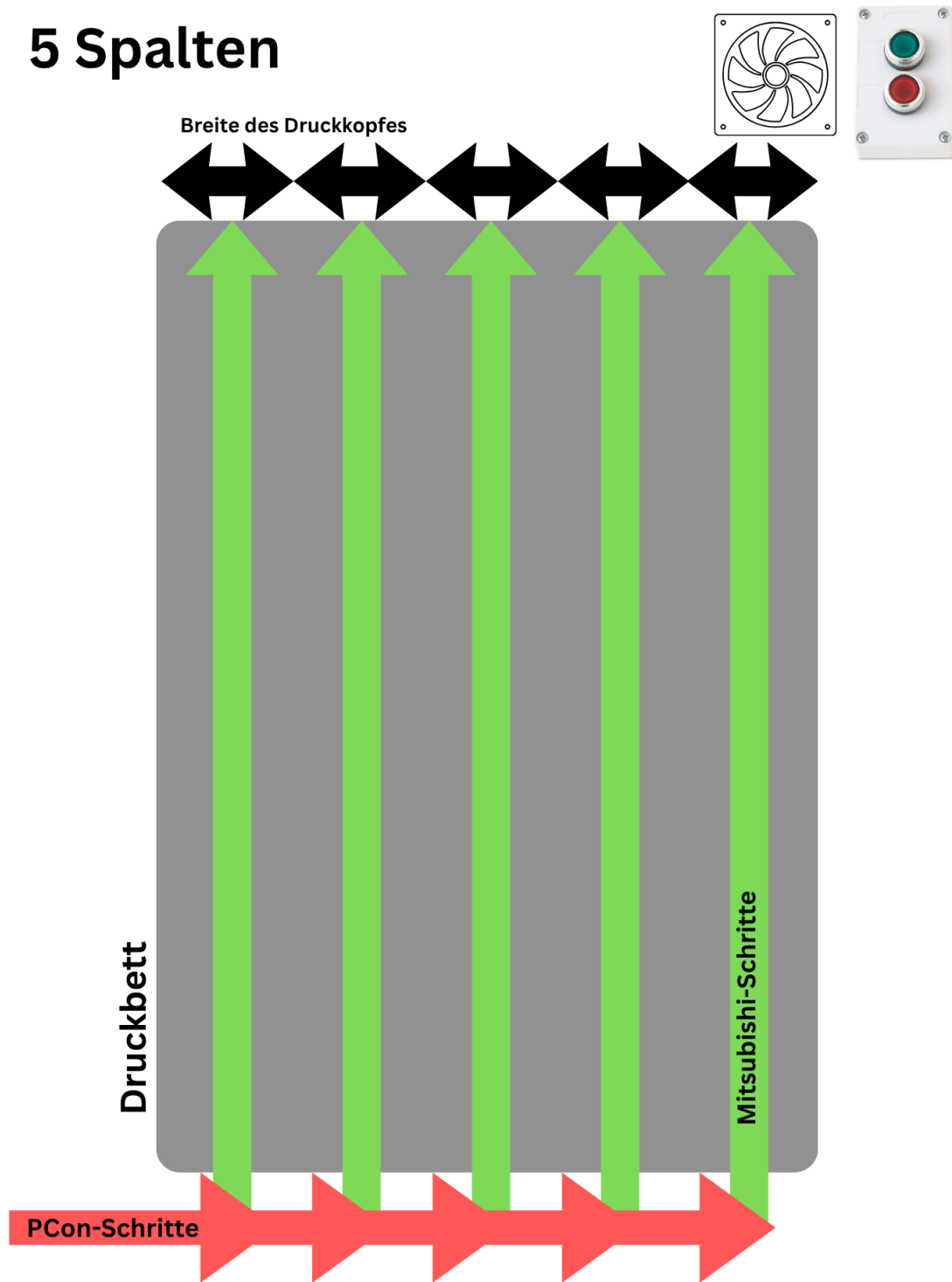


Abbildung 20: Ablauf bei 5 Spalten

Quellenverzeichnis

py-admin (2026). *Python Exevutive Explanation*. URL: <https://www.python.org/doc/essays/blurb/> (besucht am 7. Sep. 2026).
Lustig, Peter (2026). *Wissen macht Ah*. URL: https://de.wikipedia.org/wiki/Peter_Lustig (besucht am 15. Apr. 2026).

Abbildungsverzeichnis

1	pyautogui Erweiterung und Programm Parameter	9
2	Liste der Startbedingungen	9
3	Funktion Screenshots finden	10
4	Funktion Anzeige prüfen	10
5	Aktives PCon Fenster prüfen	11
6	Aktives MR Configurator Fenster prüfen	11
7	Anfang der MR Configurator Schrittkette, MR Schritt 1 und 2	12
8	MR Schritt 3,4 und 5	13
9	Initialschritt unabhängig von der Spaltenanzahl	14
10	PCon Schrittkette abhängig von der Spaltenanzahl des Testversuches	15
11	PCon Schlussschritt unabhängig von Spaltenanzahl	16
12	Checkliste zur Bestätigung aufrufen	17
13	Anzahl der Spalten abfragen	17
14	Gesamtansicht des Drucker Systems	20
15	Bauraum des Drucker Systems	21
16	Rückseite, Linearantrieb IAI (von PCon gesteuert)	22
17	Vorderseite, Linearantrieb IAI (von PCon gesteuert)	23
18	Rückseite, Linearantrieb von Mitsubishi (von MR Configurator gesteuert)	24
19	Ablauf bei 3 Spalten	25
20	Ablauf bei 5 Spalten	26

Tabellenverzeichnis

Eidesstattliche Erklärung

Ich erkläre hiermit, dass ich diese Arbeit selbstständig ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Alle den benutzten Quellen wörtlich oder sinngemäß entnommenen Stellen sind als solche einzeln kenntlich gemacht.

Diese Arbeit ist bislang keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht worden.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Leipzig, 9. September 2026

.....
Jonathan Blänsdorf

.....
Paul Felix Freiburger